

Team Detailed Steps Guide

Last Updated: 2026-04-08 **Project:** HIPAA-Compliant Home Health and Hospice AI Platform **Purpose:** Give each team member a detailed execution path so work starts in the right order and handoffs stay clear

How To Use This File

This file is not the phase tracker. It is the practical execution guide for each person.

Use it like this:

1. Read your own section first
2. Check what inputs you need from others
3. Complete your current phase in order
4. Hand over the required outputs before the next person depends on them
5. Update `PROGRESS.md` after completing major tasks

The build order for the whole project is:

```
Architecture and scope
-> secure project setup
-> upload and storage
-> OCR and extraction
-> review and validation
-> POC generation
-> risk prediction
-> integration, metrics, and final demo
```

The patient or caregiver portal is not the main build target right now.

Shared Rules For Everyone

Rule 1: Build The Core Workflow First

The first end-to-end product must support:

```
Login -> Upload document -> Extract fields -> Review and edit -> Save approved
record -> Generate draft POC -> Show risk score
```

If this workflow is incomplete, do not jump ahead into extra features.

Rule 2: Freeze Contracts Early

- Backend response formats must be agreed before frontend integration
- Extraction fields must be agreed before OCR work expands

- User roles must be agreed before auth and audit work expands

Rule 3: Work In Vertical Slices

Finish one working slice before starting the next:

- Upload working
- Then extraction working
- Then review working
- Then generation working
- Then prediction working

Rule 4: Leave Evidence

For every completed task, leave:

- a working file or implementation
- notes on what was decided
- proof of testing
- updates in `PROGRESS.md`

Abdul Rehman

Role: Team Lead, Cloud Owner, Security Owner, ML Owner

Main responsibility: Make sure the product is technically correct, built in the right order, and not allowed to drift in scope.

Abdul Phase 0: Kickoff And Scope Lock

1. Read the proposal and reduce it to the actual MVP.
2. Decide what is in scope for the first build.
3. Push the patient portal and advanced extras to a later stage.
4. Confirm the stack: `Next.js + FastAPI + PostgreSQL + Azure`.
5. Prepare the system architecture diagram.
6. Define user roles such as clinician, reviewer, and admin.
7. Define which Azure services are required and which are optional.
8. Create a weekly meeting plan and owner-based task board.
9. Tell Asad he must freeze the JSON schema and APIs before frontend integration.
10. Tell Ayesha to begin with wireframes and the frontend shell only.

Output Abdul must deliver

- Architecture diagram
- MVP scope statement
- Team ownership confirmation
- Azure service list
- Security checklist draft

Abdul Phase 1: Cloud And Environment Foundation

1. Create the cloud resource plan.
2. Decide how environments will be separated: local, test, demo.
3. Set up Azure resource group structure.
4. Decide storage, secrets, and logging design.
5. Set the rules for environment variables and secret storage.
6. Decide how the app will be deployed.
7. Define the RBAC structure at a high level.
8. Create the Docker and deployment expectations for the team.
9. Review cost-sensitive services before the team depends on them.

Output Abdul must deliver

- Environment plan
- Cloud setup plan
- Secret-handling rules
- RBAC draft
- Deployment baseline

Abdul Phase 2: Secure Upload Architecture

1. Review how documents will be uploaded and stored.
2. Decide naming and storage rules for document blobs.
3. Define encryption expectations at rest and in transit.
4. Define the audit events that must be captured.
5. Define retention assumptions for uploaded documents.
6. Review access restrictions for uploaded files.
7. Confirm that upload paths do not expose protected information.

Output Abdul must deliver

- Upload security policy
- Audit event list
- Storage access rules
- Retention assumptions

Abdul Phase 3: OCR Strategy And Extraction Governance

1. Approve the MVP form list.
2. Approve the golden set for evaluation.
3. Decide how extraction accuracy will be measured.
4. Review OCR cost and latency expectations.
5. Define fallback behavior for poor scans.
6. Check that OCR output is not treated as final truth without review.
7. Review whether field confidence values are being captured.

Output Abdul must deliver

- Accuracy measurement plan
- Golden set approval
- OCR fallback policy
- Cost and latency guardrails

Abdul Phase 4: Review Workflow Governance

1. Decide which actions must be audited.
2. Define what counts as approve, edit, and reject.
3. Confirm reviewer permissions.
4. Confirm which changes require reason or comments.
5. Approve field-level source traceability expectations.
6. Decide how reviewer time will be measured.
7. Review the screen flow with Ayesha before sign-off.

Output Abdul must deliver

- Review workflow rules
- Audit logging requirements
- Reviewer permission rules
- KPI definition for review time

Abdul Phase 5: Plan Of Care Generation Governance

1. Decide whether the model can generate only from approved structured data.
2. Define the source material used for retrieval.
3. Approve the citation requirement for generated sections.
4. Define what must be blocked as unsupported content.
5. Approve the evaluation method for draft acceptance.
6. Review cost and latency impact of the generation step.

Output Abdul must deliver

- LLM usage rules
- Citation requirement
- Guardrail rules
- Draft evaluation method

Abdul Phase 6: Risk Model Ownership

1. Decide the exact target outcome for the readmission model.
2. Define the allowed feature set.
3. Build the first baseline model instead of over-optimizing early.
4. Measure AUROC and Brier score.
5. Calibrate the probability output.
6. Add explanation support using SHAP or equivalent.
7. Document model assumptions and limitations.
8. Review how the model will be deployed.

Output Abdul must deliver

- Baseline model
- Metrics summary
- Calibration result
- Explanation strategy
- Model limitations write-up

Abdul Phase 7: Integration, Security, And Demo Readiness

1. Ensure Dockerization is complete.
2. Review the integrated workflow end to end.
3. Run the security checklist against the real implementation.
4. Verify audit logs and access restrictions.
5. Confirm latency and cost numbers.
6. Review KPI evidence from the full product.
7. Prepare the final deployment and demo sequence.

Output Abdul must deliver

- Integration sign-off
- Security review result
- KPI summary
- Demo sequence

Abdul Phase 8: Final Submission And Leadership Tasks

1. Combine all team outputs into one final story.
2. Finalize technical diagrams.
3. Review report consistency.
4. Lead final demo rehearsals.
5. Make sure every member knows what they will present.
6. Prepare for viva questions around architecture, compliance, and AI choices.

What Abdul Must Watch For

- Scope expansion
- Unclear ownership
- Frontend and backend drifting apart
- Security being postponed until the end
- Model work starting before clean structured data exists

Asad Rasheed

Role: Backend Owner, OCR Owner, API Contract Owner

Main responsibility: Build the backend system that accepts documents, extracts data, validates it, stores it, and exposes stable APIs to the frontend.

Asad Phase 0: Requirements, Fields, And Contracts

1. Collect the target OASIS and Plan of Care forms for MVP.
2. Read each form carefully and list all important fields.
3. Group fields by sections such as patient info, diagnosis, dates, mobility, and medications.
4. Mark which fields are mandatory and which are optional.
5. Define the output JSON shape for extracted data.
6. Define document status values such as uploaded, processing, extracted, in_review, approved, rejected.
7. Draft the API list for upload, extraction, review queue, save review, POC generation, and risk score retrieval.
8. Share the schema with Abdul for approval before large backend work starts.
9. Share mock responses with Ayesha so she can build UI in parallel.

Output Asad must deliver

- Field dictionary
- JSON schema
- Document lifecycle states
- API contract draft

Asad Phase 1: Backend Project Foundation

1. Create the FastAPI project structure.
2. Set up configuration handling for environments.
3. Connect PostgreSQL.
4. Add migration support.
5. Create models for users, documents, extracted fields, review actions, and generated drafts.
6. Add a health check endpoint.
7. Add basic authentication structure or placeholder.
8. Set up API docs.
9. Make sure the backend starts cleanly in Docker.

Output Asad must deliver

- Running FastAPI app
- Database schema baseline
- Health endpoint
- Docker-ready backend

Asad Phase 2: Upload And Storage Integration

1. Build the upload endpoint.
2. Validate file type and file size.
3. Send files to storage.
4. Save metadata in PostgreSQL.
5. Create the document status lifecycle.

6. Return document IDs and initial processing state.
7. Create audit records for upload actions.
8. Test upload behavior with multiple sample files.

Output Asad must deliver

- Working upload API
- Metadata persistence
- Status handling
- Upload audit logging

Asad Phase 3: OCR And Structured Extraction

1. Integrate Azure Document Intelligence or the chosen OCR service.
2. Receive OCR output and normalize it.
3. Map OCR output into the agreed JSON schema.
4. Normalize dates, identifiers, and coded fields.
5. Add field confidence values.
6. Save raw OCR text and structured extraction output.
7. Create retry and failure handling for bad scans.
8. Expose an extraction result endpoint.
9. Measure performance on the MVP forms.

Output Asad must deliver

- Working OCR pipeline
- Structured extraction output
- Extraction API
- Accuracy notes for sample documents

Asad Phase 4: Review And Validation Backend

1. Build rule checks for missing or inconsistent values.
2. Add support for approve, edit, and reject actions.
3. Save edit history and approval state.
4. Add source-to-field mapping where possible.
5. Create reviewer queue and detail endpoints.
6. Add comments or reason support for edits and rejections.
7. Add audit log entries for every review action.
8. Add reviewer-time measurement hooks if required.

Output Asad must deliver

- Review endpoints
- Validation rule engine
- Revision history
- Audit trail support

Asad Phase 5: Plan Of Care Draft Backend

1. Build the retrieval source preparation step.
2. Create template logic for care-plan structure.
3. Integrate the LLM call behind a secure endpoint.
4. Generate sections only from approved data.
5. Attach citations or source references to each section.
6. Save generated drafts and versions.
7. Add retry and regeneration logic.
8. Support clinician edits and approval persistence.

Output Asad must deliver

- POC generation endpoint
- Draft persistence
- Citation-backed output
- Draft evaluation notes

Asad Phase 6: Risk Prediction Backend

1. Create the feature extraction pipeline from approved records.
2. Build the inference endpoint.
3. Persist model output and metadata.
4. Support model versioning.
5. Expose explanation fields to the frontend.
6. Add fallback behavior when input features are incomplete.
7. Test the prediction pipeline end to end.

Output Asad must deliver

- Prediction API
- Feature pipeline
- Stored model results
- Explanation payload

Asad Phase 7: Integration And Stability

1. Run integration tests across the complete workflow.
2. Fix contract mismatches reported by Ayesha.
3. Improve error handling and status transitions.
4. Verify logs and data consistency.
5. Measure extraction accuracy and reviewer-time metrics.
6. Help prepare backend evidence for the final report.

What Asad Must Watch For

- Changing response shapes without telling Ayesha
- Growing the schema before the MVP fields are stable
- OCR outputs being saved without normalization

- Missing audit logs for reviewer actions

Ayesh Ahmed

Role: Frontend Owner, Clinician Workflow Owner

Main responsibility: Build the actual interface clinicians will use to upload documents, review fields, approve data, read care-plan drafts, and view risk scores.

Ayesh Phase 0: UX Planning And Wireframes

1. Understand the full MVP workflow.
2. List all screens needed for the MVP only.
3. Design wireframes for login, dashboard, upload, extraction results, review queue, review detail, POC draft, and risk display.
4. Confirm that the review screen supports edits and approval, not just display.
5. Review the screens with Abdul for workflow approval.
6. Review the expected response shapes from Asad.

Output Ayesh must deliver

- MVP wireframes
- Screen list
- UI flow map

Ayesh Phase 1: Frontend Project Foundation

1. Create the Next.js project structure.
2. Configure TypeScript and UI library.
3. Create app shell and navigation.
4. Create login page.
5. Create dashboard placeholder.
6. Create shared UI components for forms, cards, tables, and status states.
7. Add API client structure.
8. Make sure the frontend runs locally.

Output Ayesh must deliver

- Running frontend shell
- Shared component base
- Navigation structure

Ayesh Phase 2: Upload Workflow UI

1. Build the upload page.
2. Add file picker and validation feedback.
3. Show upload progress or submission state.
4. Show uploaded-document list or status queue.

5. Add error states for bad file types, failed upload, and network issues.
6. Integrate with Asad's upload API.
7. Verify the upload flow with sample documents.

Output Ayesh must deliver

- Upload screen
- Document status screen
- Error and loading states

Ayesh Phase 3: Extraction Results UI

1. Build the extraction results page.
2. Group fields by logical sections.
3. Show confidence or warning markers.
4. Show missing values clearly.
5. Add support for refresh or retry if processing fails.
6. Make the screen easy for a clinician to scan quickly.
7. Integrate with Asad's extraction results API.

Output Ayesh must deliver

- Extraction result screen
- Sectioned field layout
- Confidence and missing-field indicators

Ayesh Phase 4: Review And Approval UI

1. Build the reviewer queue page.
2. Build the document review detail screen.
3. Add accept, edit, and reject actions.
4. Show source snippets or field origin where available.
5. Show change history or status timeline.
6. Add comments or reason inputs if required.
7. Integrate review APIs.
8. Test the workflow with realistic scenarios.

Output Ayesh must deliver

- Review queue
- Review detail screen
- Approval actions
- Review feedback and history view

Ayesh Phase 5: Plan Of Care Editor UI

1. Build the draft care-plan page.
2. Render generated sections clearly.
3. Show citations next to generated content.

4. Add edit controls for clinicians.
5. Add warnings for uncertain or unsupported sections.
6. Support regeneration or version switching if included.
7. Integrate POC generation APIs.

Output Ayesh must deliver

- Care-plan draft editor
- Citation display
- Edit and approval flow

Ayesh Phase 6: Risk Score UI

1. Build a dashboard card or panel for readmission risk.
2. Show the risk score clearly.
3. Show a reason summary or explanation.
4. Highlight high-risk cases visually but clearly.
5. Add fallback UI when predictions are unavailable.
6. Integrate risk APIs.
7. Test whether the result is understandable to a non-technical clinician.

Output Ayesh must deliver

- Risk score card
- Explanation panel
- Clinician-friendly visual treatment

Ayesh Phase 7: Integration And Demo Polish

1. Fix mismatches between frontend and backend.
2. Improve all loading, error, and empty states.
3. Make the full workflow responsive.
4. Validate role-based access behavior.
5. Add demo-friendly polish without making the UI misleading.
6. Capture final screenshots for the report and presentation.
7. Help script the final demo flow.

What Ayesh Must Watch For

- Building extra screens that are outside MVP
- Depending on undefined backend response fields
- Hiding important review uncertainty from users
- Focusing on design polish before workflow clarity

Handoff Checklist

Abdul To Asad

Abdul must hand over:

- approved architecture
- allowed services
- security boundaries
- user-role definitions
- environment strategy

Asad To Ayesh

Asad must hand over:

- API endpoints
- request and response shapes
- field schema
- validation error shape
- mock data when endpoints are incomplete

Ayesh To Abdul And Asad

Ayesh must hand over:

- UI issues caused by missing or unclear backend fields
- workflow problems discovered during manual testing
- screens that need more audit or permission detail

What Each Person Should Finish In The First Week

Abdul

- Final MVP scope
- Architecture diagram
- Azure service decision list
- Security checklist draft

Asad

- Field dictionary
- JSON schema
- API contract draft
- Backend module plan

Ayesh

- Wireframes
 - Screen list
 - Frontend shell plan
 - Mocked UI flow
-

Final Reminder

The team should not treat this as three separate mini-projects.

It is one system with one critical dependency chain:

```
Abdul locks architecture and rules  
-> Asad defines data contracts and backend flow  
-> Ayesh builds user-facing workflow on those contracts  
-> Abdul integrates security, ML, deployment, and final sign-off
```

If that order is respected, the team will move faster and with less rework.